

Preface

Status: Definitional.

This book scaffold organizes the Simplex / Anomalon database theory program into a disciplined, implementation-oriented structure. The aim is not to replace Codd-style relational theory. The aim is to extend the engineering surface to data that moves, changes representation, becomes evidence, expires, or is claimed by an observer.

Core thesis:

Classical database theory ensures structural consistency of stored data; a complete modern theory must also ensure that every claimed fact is a replayable, observer-relative, authority-bounded statement whose failure modes are explicitly terminalized through finite obstruction.

The relational baseline is Codd-Date-Darwen predicate discipline: relation is not table, tuple is not row, attribute is not column, type is not representation, predicate is not storage, and logic is not implementation accident. The Simplex layer begins only after that baseline is respected.

Reader Contract

- Normative definitions live in chapters 00-08, `glossary/`, `spec/`, and `contracts/`.
- Absorbed source appendices are provenance, not the authority for current taxonomy.
- Experimental regimes stay quarantined unless they satisfy the protected-term checklist.
- Proof-status labels indicate how much authority a claim has.

Reading Order

1. Codd-Date-Darwen baseline.
2. Formal substrate.
3. BQNF data in motion.
4. GTMUR representation transport.
5. Epistemic / observer forms.
6. Gauge and higher data regimes quarantine.
7. Implementation contracts.
8. Open problems.

Non-Claims

- Does not replace Codd / relational theory.
- Does not claim all data problems are normal forms.
- Does not claim certification equals authorization.
- Does not claim metadata creates authority.
- Does not require category theory for implementation.
- Does not prove all experimental regimes.

Codd-Date-Darwen Baseline

Status: Definitional.

Classical relational theory is the foundation. It disciplines data at rest by requiring predicate precision, declared types, keys, dependencies, domains, joins, and constraints. Simplex / Anomalon does not replace this baseline; it extends the engineering surface to claims that move, transform, cache, replicate, expire, or become authority-bearing.

Baseline Rule

A relation is not a bag of rows. It is a predicate extension under a declared type system.

Keep these distinctions strict:

Relational concept	Implementation term to keep separate
relation	table
tuple	row
attribute	column
type	representation
predicate	storage layout
logic	implementation accident

Date/Darwen and Tutorial D matter here because they insist that a relational language should expose predicates, types, and relational operators instead of hiding them behind SQL storage idioms. Celko matters from the production side: SQL idioms, constraints, hierarchies, production modeling, and common ways implementations lose the logical predicate.

Date/Darwen:

predicate discipline, relational purity, type clarity

Celko:

SQL idioms, constraints, hierarchies, production modeling, failure patterns

Simplex Extension

Simplex normal forms start from the Date/Darwen demand for predicate precision, then extend it to distributed and agent-mediated systems where the predicate alone is no longer enough.

A modern data claim also needs:

predicate claim
observer
evidence
freshness
representation history
loss ledger
verifier
terminal route
authorization boundary

This extension is conservative. It does not weaken relational theory. It asks what must be added when clean predicates are cached, transported, observed, summarized, expired, delegated, or used to authorize action.

Non-Claims

- Does not replace Codd / relational theory.
- Does not claim all data problems are normal forms.
- Does not claim certification equals authorization.
- Does not claim metadata creates authority.
- Does not require category theory for implementation.

- Does not prove experimental regimes.

TODO

- Add a compact Tutorial D example mapping relation, tuple, attribute, type, predicate, and operator to the Simplex packet vocabulary.
- Add Celko-style SQL implementation examples showing where predicate discipline is lost in production schemas.

Formal Substrate

Status: Definitional / Mechanization target.

All certified contracts bind to the finite obstruction substrate:

$$\begin{aligned} C^{\wedge 0} &\rightarrow C^{\wedge 1} \rightarrow C^{\wedge 2} \\ Q^{\wedge 1} &= C^{\wedge 1} / \text{im}(d_0) \end{aligned}$$

Canonical symbols:

Symbol	Meaning
$C^{\wedge 0}$	repairs / witnesses / patches
$C^{\wedge 1}$	defects / mismatches / representation defects
$C^{\wedge 2}$	closure failures / higher consistency failures
d_0	repair-to-defect map
d_1	defect-to-closure map
$Q^{\wedge 1}$	$C^{\wedge 1} / \text{im}(d_0)$
ω	defect
$[\omega]$	obstruction class
Φ_i	quotient residual magnitude
Γ	closure defect magnitude
χ_i	evidence sufficiency
μ	optional policy/gauge selector
τ	terminalizer
Λ	lift/refusal discipline

Classification:

Case	Meaning	Terminal route
$[\omega] = 0$	exact repair	Repair Or Preserve

Case	Meaning	Terminal route
	exists	
$[\omega] \neq 0$ and $d1(\omega) = 0$	coherent residual	Lift, Escalate, Recompute
$d1(\omega) \neq 0$	closure failure	ClosureFailure, Refuse
$\chi_i = 0$	missing evidence	BudgetUnknown, ConservativeInvalidate, Invalidate

Implementation Reading

The substrate is not a demand that implementers use category theory. A verifier can implement this as finite packet validation, matrix/rule evaluation, and terminal-route selection.

Non-Claims

- Does not replace Codd / relational theory.
- Does not claim all data problems are normal forms.
- Does not claim certification equals authorization.
- Does not claim metadata creates authority.
- Does not require category theory for implementation.
- Does not prove all experimental regimes.

BQNF: Data in Motion

Status: Operational semantics / Mechanization target.

Boundary-Quotient Normal Form (BQNF) is the core change normal form. It disciplines CDC events, cache repair, and materialized-view repair by asking whether a write event can be applied as a certified repair to an observer boundary.

Five-Clause Checklist

1. boundary declared
2. delta computable
3. quotient class computable
4. repair image declared
5. terminal route declared

Operational Rule

```
if event misses boundary:
    Preserve
elif  $\omega = d_0(\alpha)$ :
    Repair( $\alpha$ )
elif  $X_i = 0$ :
    Invalidate or ConservativeInvalidate
elif  $d_1(\omega) \neq 0$ :
    ClosureFailure
else:
    Lift, Escalate, Recompute, or Invalidate
```


Orders / paid_revenue_by_customer

Canonical demo: [../examples/01_bqnf_orders_paid_revenue.md](https://github.com/IBM/ibm-db-warehouse/blob/master/examples/01_bqnf_orders_paid_revenue.md).

Required outcomes:

Case	Required terminal
Missing CDC status / missing predicate evidence	Invalidate
Unrelated column update	Preserve
MAX without extremum witness	Unsupported or Lift
Exact supported aggregate delta	Repair

Non-Claims

- Does not replace Codd / relational theory.
- Does not claim all data problems are normal forms.
- Does not claim certification equals authorization.
- Does not claim metadata creates authority.
- Does not require category theory for implementation.
- Does not prove all experimental regimes.

GTMUR Representation Transport

Status: Operational semantics / Conditional theorem.

GTMUR is an authority contract for representation transport. A representation may be equivalent for query evaluation while not equivalent for authority.

Core witness:

$\text{chi_F}: D_{\text{target}} F \Rightarrow \text{bar } F D_{\text{source}}$

Operational reading:

Updates commute with representation change, up to declared loss.

Required Objects

- source representation
- target representation
- transport F
- finite-core action $\text{bar } F$
- descent witness chi_F
- certificate form
- loss ledger $\text{Loss}(F)$ when distinctions are erased
- target verifier record

Failure Terminals

- `Refuse` for missing witness.
- `Quarantine` for unverified certificate packet.

- **Lift** when policy/authority must be checked externally.
- **Reject** for authority laundering.

CertifiedNF(x) does not imply **AuthorizedNF(x)**.

A certificate table without a verifier is just another table.

Non-Claims

- Does not replace Codd / relational theory.
- Does not claim all data problems are normal forms.
- Does not claim certification equals authorization.
- Does not claim metadata creates authority.
- Does not require category theory for implementation.
- Does not prove all experimental regimes.

Epistemic and Observer Forms

Status: Operational semantics / Proof obligation.

Distributed systems contain bounded claims about facts, not one omniscient global fact table. Observer / epistemic forms discipline locality, freshness, authority, context, provenance, and comparison.

Core packet:

what_is_known
by_whom
as_of_when
under_which_authority
with_what_uncertainty
terminal_route_if_challenged

Core Forms

Form	Normalizes	Anomaly excluded	Source
Location NF	logical identity and physical placement	ghost locality	Proc oblig
Replica NF	origin, epoch, causal frontier, divergence budget	false sameness	Proc oblig
State NF	value, version, context, confidence, freshness, authority	state flattening	Proc oblig
Freshness NF	read timestamp, lag, age, invalidation state, SLA	stale authority	Mech target
Authority NF	available, valid, certified, authorized, actionable	authority laundering	Mech target
Context contract	typed user, retrieval, tool, and policy layers	context smuggling	Proc oblig

Form	Normalizes	Anomaly excluded	
Observer contract	observer, measurements, equivalence, comparison envelope	observer overreach	Proc oblig
Epistemic packet	what/by whom/as of/authority/uncertainty/terminal	omniscience fiction	Proc oblig

Freshness timestamp != freshness authority. Local validity != global authority.

Observer/domain extension provenance:

[11_observer_distinguishability_domain_extensions.md](#).

Non-Claims

- Does not replace Codd / relational theory.
- Does not claim all data problems are normal forms.
- Does not claim certification equals authorization.
- Does not claim metadata creates authority.
- Does not require category theory for implementation.
- Does not prove all experimental regimes.



Gauge and Higher Data Regimes

Status: Experimental conjecture / Domain extension.

This chapter is the quarantine boundary. Experimental data regimes are allowed as operational conjectures, not as core normal forms.

Protected Term Rule

NF is a protected term. A proposal may be called a normal form only if it declares:

1. the representation normalized
2. the anomaly excluded
3. the finite packet that checks it
4. the verifier or executable check
5. the terminal routes
6. the implementation failure it prevents

If it cannot answer all six, it is not an NF. It is a contract, packet, ledger, regime, conjecture, or domain extension.

Quarantine Policy

The database program should not absorb physics or every application domain. It should export a finite-authority grammar that physics, legal, market, satellite, condensed-matter, and other applications may instantiate later.

The following do not enter the core taxonomy:

- Bell NF
- Bohm NF
- Copenhagen NF

- DeWitt-Everett NF
- Wheeler NF
- UnGodLy NF
- Quantum NF
- Market NF
- Legal NF
- Satellite regimes
- condensed-matter regimes

Allowed namespaces:

Namespace	Use
<code>contracts/experimental/</code>	speculative observer/ontology/data-regime contracts
<code>contracts/domain/</code>	domain-specific application regimes, if added later
<code>docs/folklore/</code>	comedy or folklore diagnostics, if added later

Comedy or folklore names may be retained only as non-normative diagnostic labels. They must not appear in the core taxonomy or schema enum as normal forms.

Experimental Data Regimes

- data under gauge
- data under observation
- data under delegation
- data under adversary
- data under compression
- data under forgetting
- data under simulation
- data under learning
- data as field
- data as section

- data as process
- data on data

Canonical quarantined contracts:

- [../contracts/experimental/nf.experimental.gauge.yaml](#)
- [../contracts/experimental/nf.experimental.bell_contextuality.yaml](#)
- [../contracts/experimental/nf.experimental.quantum_observer_ontology.yaml](#)

Observer/domain-extension appendix:

[11_observer_distinguishability_domain_extensions.md](#).

Observer-Ontology Patterns

These are contracts for how a system is allowed to promote local observer packets into claims about hidden state, measurement, branch structure, or information ontology.

They are not physics claims. They do not say Bohm is true, Copenhagen is true, Everett is true, Bell settles ontology, or Wheeler's information ontology is physically established.

Serious thesis:

Quantum interpretations are representation regimes. Their danger is not that they are wrong; it is that they promote finite observer packets into ontological claims without declaring the transport, loss, branch, hidden-state, or realization ledger.

Patterns:

Pattern	Blocks
BellCNF observer contract	local packets promoted into illicit global state

Pattern	Blocks
Bohm hidden-state ledger	hidden variables without projection/dynamics/nonlocality ledger
Copenhagen measurement boundary	unmeasured properties promoted into measurement authority
DeWitt-Everett branch ledger	branch decomposition promoted into unledgered ontology
Wheeler realization boundary	bits/information promoted into physical authority without realization map

BellCNF is intentionally not named `BCNF_qm`; BCNF is reserved for Boyce-Codd Normal Form in relational contexts.

Domain Regimes, Not Core NFs

For satellites, use:

- orbital observation contract
- telemetry authority packet
- sensor-fusion loss ledger
- calibration residual contract

For condensed matter, use:

- finite measurement regime
- phase-obstruction packet
- order-parameter representation contract
- effective-theory loss ledger

Current domain contract examples:

- [../contracts/domain/contract.motion.locality_bound.yaml](#)
- [../contracts/domain/contract.spacetime.lightcone_causality.yaml](#)
- [../contracts/domain/contract.time.clock_sync.yaml](#)

Non-Claims

- Does not replace Codd / relational theory.
- Does not claim all data problems are normal forms.
- Does not claim certification equals authorization.
- Does not claim metadata creates authority.
- Does not require category theory for implementation.
- Does not prove all experimental regimes.

Implementation Contracts

Status: Mechanization target.

The machine-readable contract system lives in `/contracts`. Every concrete YAML contract must include:

- `id`
- `title`
- `version`
- `status`
- `category`
- `normalized_object`
- `anomaly_excluded`
- `required_declarations`
- `required_evidence`
- `obstruction_substrate`
- `validation_rules`
- `terminal_routes`
- `certificate_requirements`
- `authorization_boundary`
- `proof_obligations`
- `non_claims`
- `examples`

Core schema:

- [../contracts/normal_form_contract_schema.yaml](#)
- [../contracts/obstruction_substrate_schema.yaml](#)

Core contracts:

- [../contracts/nf.change.bqnf.yaml](#)
- [../contracts/nf.transport.gtmur.yaml](#)
- [../contracts/nf.epistemic.freshness.yaml](#)
- [../contracts/nf.epistemic.authority.yaml](#)

- [../contracts/nf.epistemic.observer.yaml](#)
- [../contracts/authority_observer_stack.yaml](#)

Quarantined contracts:

- [../contracts/experimental/nf.experimental.gauge.yaml](#)
- [../contracts/experimental/nf.experimental.bell_contextuality.yaml](#)
- [../contracts/experimental/nf.experimental.quantum_observer_ontology.yaml](#)

Domain contracts:

- [../contracts/domain/contract.motion.locality_bound.yaml](#)
- [../contracts/domain/contract.spacetime.lightcone_causality.yaml](#)
- [../contracts/domain/contract.time.clock_sync.yaml](#)

Implementation Priorities

1. YAML shape validation.
2. Canonical terminal validation.
3. BQNF Orders verifier.
4. GTMUR transport certificate checker.
5. Loss ledger checker.
6. Freshness packet checker.
7. Authority boundary checker.

Non-Claims

- Does not replace Codd / relational theory.
- Does not claim all data problems are normal forms.
- Does not claim certification equals authorization.
- Does not claim metadata creates authority.
- Does not require category theory for implementation.
- Does not prove all experimental regimes.

Open Problems

Status: Proof obligation / Mechanization target.

Proof Obligations

- BQNF exact repair soundness for supported SPJA aggregate lanes.
- No silent repair when $x_i = 0$.
- GTMUR transport soundness for SQL-to-SPJA compilation under declared loss.
- Authority gate soundness: available, valid, certified, authorized, actionable.
- Observer comparison envelopes for local-to-global claims.

Executable Tests Needed

- YAML schema validation.
- Terminal route validation.
- BQNF Orders positive/negative cases.
- GTMUR rejects missing `Loss(F)` when distinctions are erased.
- Freshness timestamp without invalidation authority fails.
- Certificate table without verifier fails authorization.

TODO Ranked by Implementation Value

1. Write a recursive contract validator for required fields.
2. Add canonical terminal-route checker.
3. Add BQNF Orders verifier fixture.
4. Add GTMUR transport fixture with missing loss ledger negative case.

5. Add freshness packet fixture.
6. Add authority boundary fixture proving `certified_implies_authorized: false`.
7. Add link checker for internal Markdown links.
8. Add archived-source disclaimer checker for absorbed appendices.

Non-Claims

- Does not replace Codd / relational theory.
- Does not claim all data problems are normal forms.
- Does not claim certification equals authorization.
- Does not claim metadata creates authority.
- Does not require category theory for implementation.
- Does not prove all experimental regimes.

Certified Normal Forms, Representation Transport, and Finite Authority for Simplex Databases

Status: Archived source / Domain extension.

Absorbed full atlas copy from

`omega_engine/research/in_work/simplex_cosmos_papers/SMPLX_009_Simplex Databases/crap/1_normal_forms.md`. The source folder was integrated into the root scaffold and removed.

Non-normative archive: current definitions are controlled by

`book/00_prelude.md` through `book/08_open_problems.md`, `glossary/`, `spec/`, and `contracts/`. If this appendix conflicts with the cleaned taxonomy, the cleaned taxonomy wins.

Status: working atlas (survey + pointers; not a single publishable theorem paper)

Scope: shape normal forms (data at rest), BQNF and motion forms (data in motion), certificate/evidence forms, authority transport, epistemic normal forms (distributed claims)

Base ontology: `0_relvar_relation_table_and_boundary_relvar.md` — what is normalized, repaired, transported, authorized.

Machine contracts: `normal_form_contracts/` — typed YAML per NF;

canonical substrate = finite obstruction cochain complex ($c_0 \rightarrow c_1 \rightarrow Q_1$); graphs/matrices/presheaves are optional `carrier_realization / geometry_module` only.

Companion papers: `000_SmplCache Simplex In-Memory Database and CDC.md`, `000_Quotient Normal Form and Boundary-Certified Databases.md`

Planned trilogy split (editorial):

Paper	Working title	Primary content from this atlas
A	<code>1a_bqnf_incremental_repair.md</code> — Boundary-Quotient Normal Form for Incremental Database Repair	§3 BQNF, terminal routes, running example, minimal category theory
B	<code>1b_gtmur_representation_transport.md</code> — Certified Representation Transport for Database Systems	§4 certificates, loss ledgers, GTMUR transport, Compilation NF (CpNF)
C	Finite Authority and Shadow Semantics	GTTC, cofinal towers, authority descent (philosophical layer)

Abstract

Classical database normal forms discipline **data at rest**: they decompose relations so dependency anomalies do not appear as accidental redundancy. Simplex database architectures discipline **data in motion, representation, and observation**: they decide whether a CDC event, cache update, query rewrite, representation change, or epistemic observation preserves declared authority, requires a loss ledger, or must terminalize via named routes (such as repair, invalidation, quarantine, or decomposition).

The primary operational components are **Boundary-Quotient Normal Form (BQNF)** for incremental repairs on data in motion, **Gauge Normal Form (GNF)** for representation changes, and the **GTMUR** transport verifier.

0. Thesis

Classical normal forms prevent structural anomalies in stored relations. Simplex normal forms prevent authority anomalies in data that moves, transforms, replicates, summarizes, delegates, or becomes evidence for action.

Rather than expanding normal forms indefinitely, this framework organizes database and observer actions into **four post-Codd data states** and a cross-cutting metadata layer:

1. Data at Rest (Codd Territory)

Normalizes static representation shapes to prevent static dependency and logical anomalies:

- Relational schemas, dependencies, keys, constraints, and stored relations (1NF–6NF, DKNF).
- Selection-projection-join (SPJ/SPJA) canonical query shapes.
- Safe tuple/domain relational calculus ranges.

2. Data in Motion (BQNF Territory)

Normalizes runtime transitions, cache updates, and incremental modifications:

- Change Data Capture (CDC) events and streams.
- View and cache repair programs.
- Boundary fingerprints (∂Q) and delta execution.

- Typed terminal routes (Repair , Invalidate , Quarantine , Decompose).

3. Data in Representation (GTMUR/Gauge Territory)

Normalizes representation shifts, compilations, and structural translations:

- Query rewrites and schema migrations.
- **Gauge Normal Form (GNF)**: The certified condition that representation freedom (e.g., JSON ↔ Protobuf, view ↔ materialized cache, record ↔ embedding) has not changed authority-relevant content.
- Loss ledgers tracking erased attributes, precision, or layout.
- Descent witnesses (χ_F) for GTMUR transport.

4. Data under Observation (Epistemic Territory)

Normalizes bounded claims about facts in distributed, cached, or agent-mediated systems:

- Visibility, freshness, and temporal bounds.
- Causal frontiers, divergence budgets, and replication epochs.
- Uncertainty scales and source provenance.
- Authority status gates (available, valid, certified, authorized, actionable).

Cross-Cutting Layer: Data on Data

The audit and governance substrate verifying the four data states:

- Metadata catalogs and policy rules.
 - Transition certificates and loss ledgers.
 - Trace logs and verifier execution records.
-

1. Classification Categories

To prevent taxonomy bloat, database structures and contracts are organized into distinct categories rather than treating every invariant as a new normal form:

- **Normal Forms:** Decidable representation shapes excluding a declared anomaly (e.g., 1NF/3NF for shape, BQNF for change, GNF for gauge).
- **Certified Forms:** Normal forms combined with a certificate, an obstruction class, and a terminal route (certified $\not\rightarrow$ authorized).
- **Authority Contracts:** Verifier and policy specifications that evaluate credentials, lineage, and permission (e.g., GTMUR transport contract, GTLA verifier).
- **Observer Contracts:** Visibility and coordinate envelopes (e.g., comparison envelopes $\Omega_{\mathfrak{O}}$).
- **Ledger Forms:** Structured accounts of lost representation, precision, or attributes (e.g., loss ledger).
- **Domain Extensions:** Extensible modules quarantined from the core database papers (e.g., relativistic coordinate frames, noisy channel boundaries, horizon boundaries, locality cones).

The concrete atlas begins below; distributed claim forms are collected in §7.

2. Shape Normal Forms

Shape normal forms enforce constraints on static schema design or algebraic query representations.

2.1 Classical Relational Normal Forms

Classical relational normal forms are static dependency disciplines. Under the Simplex databases framework, classical anomalies are treated as labeled quotient obstruction slices:

[$\boxed{\text{Declared dependency violation}} \rightarrow$
 $[q_{\text{dep}}] \neq 0 \text{ in a labeled quotient slice. }]$

Normal Form	Preserves / Enforces	Obstruction Slice
1NF	atomic values in relations	hidden nested structure, untyped repeated groups
2NF	no partial dependency of non-key facts on part of a candidate key	partial-key dependency
3NF	no transitive dependency from keys to non-key facts	transitive dependency
BCNF	every nontrivial determinant is a superkey	non-superkey determinant
EKNF	elementary key dependencies are disciplined tightly	elementary-key anomaly
4NF	nontrivial multivalued dependencies have superkey determinants	multivalued dependency
ETNF	join dependencies implied by elementary keys are disciplined	tuple-generating redundancy
5NF / PJNF	nontrivial join dependencies are key-implied	join-dependency anomaly
DKNF	all constraints follow from domains and keys	non-domain/key constraint leakage
6NF	relations decomposed into irreducible facts	over-fragmentation or reconstruction burden

2.2 SPJ and SPJA Shape Forms

Selection-projection-join (and aggregation) forms normalize query and view representations:

$$[\text{SP Form:} \quad Q \equiv \pi_A(\sigma_{\varphi}(R_1 \Join \dots \Join R_n))]$$

$$[\text{SPJA Form:} \quad Q \equiv \gamma_{G, a_1, \dots, a_k} \pi_A(\sigma_{\varphi}(R_1 \Join \dots \Join R_n))]$$

SPJA form forces aggregate evidence to be explicit, making incremental repair sound relative to declared sufficient statistics:

Aggregate	Repair Evidence Required
SUM	old value, new value, group key, predicate entry/exit
COUNT	old row membership, new row membership, group key
AVG	SUM and COUNT sufficient statistics
MIN / MAX	auxiliary extremum state or conservative invalidation
percentile / median	order-statistic state or refusal/lift

2.3 Safe Relational Calculus Forms

Relational calculus shapes support safety under the declared assumptions by enforcing that logical formulas range exclusively over finite, replayable database carriers:

- **Safe Tuple Relational Calculus (STRC):** Form: $\{t \mid \varphi(t)\}$, where φ is range-restricted. Unsafe formulas are not authority-bearing because they may range over infinite or undeclared domains.
- **Domain Relational Calculus (DRC):** Form: $\{(x_1, \dots, x_n) \mid \varphi(x_1, \dots, x_n)\}$, isolating domain variables for precise attribute masking and loss ledger accounting.

- **Prenex / Guarded Normal Form:** Form: $Q_1x_1 \cdots Q_nx_n; G(x_1, \dots, x_n) \wedge \psi$, where the guard G is a finite relation preventing the smuggling of continuum assumptions.

2.4 Datalog / Fixpoint Normal Form

Exposes recursive query structures as least fixpoints over a finite lattice:

$[\boxed{ \operatorname{lfp}(T_P) }]$

Non-monotone recursion requires stratification or refusal:

Recursion Type	Normal Form
positive recursion	least fixpoint
negation	stratified fixpoint
aggregation in recursion	monotone aggregate or refusal
nonterminating recursion	budget terminal

3. Change Normal Forms

Change normal forms discipline data in motion, tracking how incremental updates affect relational representations.

Paper A (canonical): `1a_bqnf_incremental_repair.md` — standalone BQNF paper for incremental repair, CDC, and materialized views. The subsections below are a short atlas index; prefer Paper A for publication draft.

3.0 Running example: Orders and paid_revenue_by_customer

Carry one schema through the rest of this section:

```
Orders(order_id, customer_id, amount, status)
```

Shape at rest (3NF sketch). Facts about customers belong in

`Customers(customer_id, ...)`; order amounts and status stay on `Orders`. A transitive dependency `order_id → customer_id → customer_name` is removed by decomposition — classical NF excludes the anomaly from the stored shape.

View in motion (SPJA). Materialized aggregate:

```
paid_revenue_by_customer(customer_id) :=
  SELECT customer_id, SUM(amount) AS revenue
  FROM Orders
  WHERE status = 'paid'
  GROUP BY customer_id
```

Boundary fingerprint $\$ \backslash \text{partial } Q \$$:

```
relations:  [Orders]
attributes: [customer_id, amount, status]
predicates: [status = 'paid']
groups:     [customer_id]
aggregates: [SUM(amount)]
```

CDC event $\$e\$$: `UPDATE Orders SET amount = 120, status = 'paid' WHERE order_id = 42`
(old: amount=100, status='pending', customer_id=7).

Step	Question	This event
1. Boundary	Does $\$e\$$ intersect $\$ \backslash \text{partial } Q \$$?	Yes — amount, status
2. Delta	Is $\$ \backslash \Delta Q(e) \$$ computable from CDC evidence?	Yes — predicate entry (pending → paid), group key 7, old/new amount
3. Quotient class	Is the defect exact or residual?	Exact repair: increment revenue for customer 7 by \$+20\$
4. Repair image	Witness $\$ \backslash \alpha \$$ with $\$ \backslash \omega =$	Local aggregate update (SUM lane)

Step	Question	This event
	$d_0\alpha$?	
5. Terminal route	If not exact, which route?	$\text{Repair}(\alpha)$ — not Invalidate / Escalate

If the CDC stream omitted `status` before/after, the same boundary intersection would hold but clause (2) would fail $\rightarrow$ terminal route `Invalidate` (missing evidence), not silent repair. That is the systems point BQNF is trying to capture.

3.1 Boundary-Quotient Normal Form (BQNF)

Boundary-Quotient Normal Form is the unified certified normal form for runtime database updates. It integrates boundary declarations, delta programs, and quotient classification to verify whether an update represents a sound repair or an obstruction.

Quotient obstruction (operational reading)

Fix a **layer** (one materialized view, cache entry, or observer). Declare:

- **0-cells** C^0 : admissible repair potentials (e.g., group-key deltas, predicate entry/exit adjustments, sufficient-statistic updates).
- **1-cells** C^1 : **defects** ω — finite packets describing how a write event disagrees with the current certified state relative to ∂Q .
- **Boundary operator** $d_0: C^0 \rightarrow C^1$: maps a repair potential α to its induced defect $d_0\alpha$.

Two defects are **equivalent for repair** when they differ by an admissible repair:

$$[\omega_1 \sim \omega_2; \rightarrow; \omega_1 - \omega_2 \in \text{operatorname{im}}(d_0).]$$

The **quotient obstruction space** is the space of defect classes:

$[Q^1 = C^1 / \text{operatorname{im}}(d_0), \sqquad [\omega] = 0 ; \text{Leftrightarrow}; \omega = d_0 \alpha \text{ for some admissible } \alpha.]$

Operationally: $[\omega]=0$ means “incremental repair preserves declared query semantics under ∂Q ”; $[\omega] \neq 0$ means “residual obstruction — escalate, lift auxiliary state, or invalidate.” The cochain condition $d_1 \omega = 0$ (when used) separates **closed residuals** from **closure failure**; see the terminal table below.

Theorem-shaped (Boundary exactness criterion).

Let Q be a view with declared boundary ∂Q and admissible repair potentials C^0 . If a write event e induces $\omega(e) \in C^1$ and there exists computable evidence yielding a witness α such that $\omega(e) = d_0 \alpha$, then incremental repair along α preserves the semantics of Q on the declared SPJA fragment (SUM/COUNT/AVG lanes under sufficient statistics; MIN/MAX only with declared auxiliary state or conservative invalidation).

Proof obligation: fix explicit C^0, C^1, d_0 for the workload fragment and show α updates sufficient statistics iff the view homomorphism commutes with the CDC-evidence monoid. Deferred to Paper A; SmpIcache instantiates the SUM/COUNT/AVG case.

An update event ω in C^1 satisfies BQNF under five strict subclauses:

1. **Boundary Declared:** The query/view shape must declare its boundary fingerprint: $[\partial Q = (\text{relations}, \text{attributes}, \text{predicates}, \text{groups}, \text{aggregates}, \text{policies}).]$
2. **Delta Computable:** The write event must admit a computable delta program: $[\Delta Q(e) = Q(S + \Delta_e) - Q(S).]$
3. **Quotient Class Computable:** The event defect $\omega(e)$ must map to a computable class $[\omega(e)]$ in the quotient obstruction space $Q^1 = C^1 / \text{operatorname{im}}(d_0)$.
4. **Repair Image Declared:** A sound repair is possible if and only if $[\omega(e)] = 0$ (implying the change is an exact boundary $\omega =$

$d_0\alpha$, resulting in a repair action α).

5. **Terminal Route Declared:** If $[\omega(e)] \neq 0$, the system must execute a declared terminal route:

BQNF Classification	Condition	Terminal Route
Exact Repair	$\omega = d_0\alpha$	<code>Repair(α)</code>
Closed Residual	$d_1\omega = 0, [\omega] \neq 0$	<code>Residual / Escalate</code>
Closure Failure	$d_1\omega \neq 0$	<code>ClosureFailure</code>
Unknown Obstruction	missing evidence	<code>Invalidate / BudgetUnknown</code>

Other terminal routes include: `Serialize`, `Recompute`, `Lift`, `Unsupported`, `ConservativeInvalidate`, `Quarantine` (isolate the packet because neither repair nor authorization is safe), and `Decompose` (split a compound packet into separately certifiable subpackets).

4. Evidence and Certificate Forms

These structures normalize proof and audit artifacts rather than active relations.

Paper B (canonical): `1b_gtmur_representation_transport.md` — GTMUR transport, loss ledgers, comparison envelopes, verifier architecture. The subsections below are a short atlas index.

4.1 CDC Evidence Form

Defines the canonical, structured representation of raw database write events:

```
write event
-> old row
-> new row
-> changed attributes
-> predicate membership before/after
-> group key before/after
-> policy context
```

4.2 Certificate Form

Certifies that a representation change, query rewrite, or repair step is authority-preserving. Every certificate must match this canonical packet shape:

```
{
  "certificate_id": "cert_xyz123",
  "source_regime": "sql_query_or_schema",
  "target_regime": "boundary_repair_packet",
  "morphism_kind": "project|refine|rewrite|approx",
  "preserved_skeleton": ["attributes", "keys"],
  "erased_distinctions": ["phases", "indices"],
  "loss_ledger_ref": "ledger_abc789",
  "obstruction_class": "[omega]",
  "terminal_route": "Repair|Invalidate|Lift|Escalate"
}
```

4.3 Loss Ledger Form

Documents lost resolution, forgotten attributes, or structural approximations during a representation change, preventing silent authority degradation.

4.4 Provenance Form

Normalizes the tracking of query tuples mapped to algebraic semirings ($\$t \mapsto p_t \text{ in } K\$$) to verify data lineage and policy adherence.

5. Authority Contracts

Authority contracts are not normal forms. They are active verification rules that govern whether normal forms can claim active authority.

5.1 GTMUR Transport Contract

Governs representation changes ($F: A \rightarrow A_{\mathrm{nf}}$). Authority transport is supported if and only if the system declares a descent-compatibility certificate χ_F and a loss ledger:

$$[\boxed{ \chi_F: D_{\{A_{\mathrm{nf}}\}} F \rightarrow \bar{\{F\}} D_A }]$$

GTMUR Failure Terminals:

- **TransportViolation** : No χ_F exists for the normalizing map.
- **SilentMutation** : A protected structural skeleton was altered without a ledger.
- **LossLedgerMissing** : Lossy normalization was executed without an accounting ledger.
- **TargetTerminalBypass** : The normalized object bypassed its target verifier.
- **AuthorityLaundering** : A representationally normal form is mistaken for an authorized form.

5.2 Comparison envelope ($\Omega_{\mathfrak{O}}$)

Enables different candidate normal forms or plan decompositions to be combined or compared via pullback, pushout, or limit constructions (formal details: Paper B §5):

$$[N_1(A) \xleftarrow{p_1} C(A) \xrightarrow{p_2} N_2(A)]$$

- **Pullback Policy-Data Envelope**: Pulls back data representations with policy domains.

- **Pushout Migration Envelope:** Merges schemas under declared identifications.
- **Weak Terminal Comparison Envelope:** Evaluates candidate repairs against a single terminal criterion.
- **Failure terminals:** `NoComparisonEnvelope`, `NonComposableCertificate`, and `CompletionWithoutDescent`.

5.3 Finite-shadow authority criterion (GTTC)

Governs continuous or evolving systems by asserting that a continuous object is authority-bearing if and only if there exists a cofinal, coherent tower of authorized finite shadows:

[$\exists \Lambda_0 \subseteq \Lambda \text{ cofinal} : \forall \lambda \in \Lambda_0, \text{Auth}\{\Omega(A_\lambda)\} \downarrow \chi_{\lambda\mu} \downarrow$]

- **GTTC Failure Terminals:** `NoFiniteShadow`, `NoCofinalAuthorizedShadow`, `ShadowIncoherence`, `ApproximationLedgerMissing`, and `LimitMembershipNotAuthority`.

5.4 Terminal authority verifier (GTLA)

The outermost verification engine that evaluates the entire pipeline's descent to finite, authorized packets. It is the judge of normal forms, not a member of the normal-form atlas (Paper C):

[$\Pi^\sharp = \tau^\sharp \circ_K \text{operatorname{FOC}}^\sharp \circ_K D^\sharp \circ_K \rho^\sharp \circ_K \text{mathcal{R}}^\sharp$]

6. Motion Accounting Contracts (Archived Draft)

These forms cover data that is moving but not fully accounted for by BQNF's single-boundary repair question. They are **draft finite contracts**, not physics claims. Each one must bind to a finite carrier, emit a terminal route, and state explicit non-claims before promotion.

Contract	Minimum Declaration	Anomaly Excluded	Carrier
Flow Control Contract (FCC)	ordering key, offset, watermark, capacity, backpressure, drop and replay policy	silent motion loss — stream loss, lag, duplicate, reorder, or overflow presented as exact delivery	<code>ordered_stream</code>
Light-Cone Causality Contract (LCCC)	event location/time, clock basis, propagation medium, max speed, path/latency bound	impossible causal order — event B claims dependence on A before A could reach B	<code>finite_metric</code>
Clock-Synchronization Contract (CSC)	clock id/type, sync source, drift/offset bounds, monotonicity, reset policy	exact global time fiction — wall-clock timestamps used as total order	<code>finite_metric</code>

Contract	Minimum Declaration	Anomaly Excluded	Carrier
		without uncertainty	
Locality-Bound Propagation Contract (LBPC)	finite locality graph, metric, interaction radius, velocity/decay bound, event support, observation window	instant influence — local event treated as globally available without cone or residual ledger	finite_metr
Horizon Emission Contract (HEC)	horizon boundary, interior/exterior partition, emission channel, coarse graining, conservation/loss ledger	unledgered horizon crossing — exterior emitted summary treated as interior authority	boundary_ch

6.1 Flow Control Contract (FCC)

A stream lane satisfies the **Flow Control Contract** iff each moving record can be replayed or refused through:

```
event_identity, ordering_key, offset, watermark_rule,
capacity_budget,
backpressure_policy, drop_policy, replay_window,
duplicate_handling_rule
```

FCNF is the motion discipline for queues, CDC logs, compaction lanes, and retry buffers. It complements BQNF: FCNF proves the event lane did not silently lose or reorder evidence; BQNF proves what a surviving event may do to an observer boundary.

6.2 Light-Cone Causality Contract (LCCC)

A distributed event claim satisfies **LCCC** iff a claimed causal edge is admissible under declared propagation bounds:

```
event_location, event_time, reference_frame_or_clock_basis,  
propagation_medium, max_signal_speed, path_model, latency_bound
```

If two writes are outside each other's declared cone, the system may terminalize as `SpacelikeConcurrent` or require `ConsensusRequired`; it may not silently impose a total order.

6.3 Clock-Synchronization Contract (CSC)

A timestamp claim satisfies **CSC** iff it carries clock authority and uncertainty:

```
clock_id, clock_type, sync_source, drift_bound, offset_bound,  
monotonicity_rule, reset_policy, total_order_authority_rule
```

The contract excludes treating a timestamp scalar as exact global time. When uncertainty intervals overlap, total ordering requires a clock-bound proof or consensus certificate.

6.4 Locality-Bound Propagation Contract (LBPC)

LBPC imports the operational shape of a Lieb-Robinson-style finite locality bound into data systems: a local perturbation may affect distant observers only inside a declared locality cone, or outside it with an explicit residual/suppression ledger.


```
finite_locality_graph, graph_metric,  
local_interaction_or_dependency_radius,  
velocity_bound, decay_bound_or_epsilon, event_support,  
observation_window
```

This is useful for dependency graphs, distributed invalidation, lattice shadows, network neighborhoods, and bounded-effect agent tools. It does **not** prove a physical theorem for undeclared systems.

6.5 Horizon Emission Contract (HEC)

HEC accounts for data crossing a boundary after which the original interior state is inaccessible to an exterior observer: redaction, eviction, compaction, lossy summarization, tombstoning, or a Hawking-radiation-style emission channel.

```
horizon_boundary, interior_region, exterior_observer, crossing_rule,  
emission_channel, coarse_graining_map, conservation_ledger,  
recoverability_class, refusal_rule_for_interior_claims
```

The finite rule is simple: exterior packets can authorize only what the emission channel and loss/conservation ledger support. Row-level or microstate claims across the horizon require a lift witness or must terminalize as

```
InteriorInaccessible , RecoverabilityUnknown , or  
RefuseUnledgedHorizon .
```

7. Epistemic Verification and Distributed Claims

Classical Codd-style database theory assumed a single coherent store: *the database contains facts*. Distributed, cached, and agent-mediated systems require a different default:

```
[ \boxed{ \text{The system contains bounded, authorized \textbf{claims} about  
facts.} } ]
```

Rather than treating every metadata boundary as a new normal form, we organize checking disciplines into the four post-Codd data states and define a minimal basis of ten core anomalies.

Core Anomaly and Form Mapping

Data State	Checking Discipline	Minimum Declaration	Anom Exclu
Data at Rest	Shape NFs (1NF–3NF)	Functional dependencies, candidate keys	redunda anomali
Data in Motion	Boundary-Quotient NF (BQNF)	boundary, delta, quotient class, repair image, terminal route	unsound repair
	Flow Control NF (FCNF)	ordering key, offset, watermark, capacity, drop/replay policy	silent m loss — q or CDC la hides los duplicate reorder, overflow
	Locality/Causality contracts (LCCC, LBPC)	finite metric graph, event support, propagation/time bounds	instant influenc local even treated a globally available without f cone or residual l
	Horizon Emission Contract (HEC)	horizon boundary, emission channel,	unledge horizon crossing exterior summary

Data State	Checking Discipline	Minimum Declaration	Anom Exclu
		loss/conservation ledger	promote interior authority
Data in Representation	Gauge Normal Form (GNF)	source/target format, authority skeleton, conversion witness	gauge di represent differ in authority content
	GTMUR Transport	descent witness χ_F , loss ledger	schema residual privacy - silent loss of structure
Data under Observation	Location NF (LNF)	logical ID, physical placement, movement policy	ghost loc — logical location from active copies
	Replica NF (RNF) / FNF	origin, epoch, causal frontier, freshness SLA	staleness fraud — serving state with lag declaration
	Orchestration NF (ONF)	initiator, tool, input/output, delegation, rollback	delegation overreach actions beyond delegative boundaries
	Context NF (CNF)	typed prompt, retrieval, tool, and policy layers	context smuggling prompt injection merged contexts

Data State	Checking Discipline	Minimum Declaration	Anom Exclu
	Provenance NF (PvNF) / ANF	source facts, model/tool calls, seeds, signatures	authorit launders unverifie claims promote facts
	Observer NF (ODNF)	measurements, equivalence relations, transport	observe — cross- observer informat exposure
Data on Data	Certificate Form	certificate ID, source/target, preserved skeleton	orphan certifica certificate exists wit verifier cl
	Verifier Record	verifier ID, replay logs, signatures	verifier v certificate logged b never rep checked

A certificate table without a verifier is just another table.

If a certificate exists (proving *certification*), but no active verifier replay checks it (yielding *authorization*), the system is in a **verifier void**.

(See §7.9 — gates are **not** implied by one another.)

7.1 Location Normal Form (LNF)

A datum is in **Location Normal Form** iff every authoritative reference declares:

`logical_id, physical_placement, replica_set, storage_tier,
relocation_policy`

Use: federated query routing, cache pinning, tiered storage, vector/GPU offload. **Certification:** placement certificate when data moves (GTMUR transport + LNF ledger entry).

7.2 Replica Normal Form (RNF)

A copy is in **Replica Normal Form** iff it carries:

`origin, epoch, causal_frontier, divergence_budget,
reconciliation_rule`

Use: multi-region SQL, CRDTs, read replicas, event sourcing. **Terminal routes:** `Repair, Serialize, Recompute, Escalate` when frontier or budget is exceeded (aligns with BQNF §3.1).

7.3 State Normal Form (StNF)

A record is in **State Normal Form** iff its state is not a bare scalar but a packet:

`(value, version, causal_context, confidence, freshness, authority)`

Composition with BQNF: BQNF certifies *how* a write updates a layer; StNF certifies *what* consumers read at rest between writes. A BQNF `Repair( $\alpha$ )` should update the StNF packet in lockstep (version bump, freshness, authority bit).

Example (Orders): not `revenue = 42000` but:

```
{
  "value": 42000,
  "version": "mv_paid_revenue_v17",
  "causal_context": "cdc_evt_42",
  "confidence": 1.0,
  "freshness": { "read_at": "2026-05-19T12:00:00Z", "source":
"smpcache_primary" },
  "authority": "certified_not_authorized"
}
```

7.4 Orchestration Normal Form (ONF)

An agent or workflow step is in **Orchestration Normal Form** iff each action declares:

```
initiator, tool, input_state, output_state, delegated_authority,
rollback_path, terminal_route
```

Use: LLM tool loops, ETL DAGs, human-in-the-loop approval. **Analogy:** ONF is BQNF for *agent actions* — same terminal discipline (*Repair / Refuse / Escalate*), different boundary (tools and policies, not SQL aggregates).

7.5 Context Normal Form (CNF)

Agent memory is in **Context Normal Form** iff prompts, retrieved context, tool results, and policy constraints occupy **typed layers** with explicit merge rules — never a single undifferentiated string.

```
user_intent_layer
retrieval_layer (cite source_id per chunk)
tool_output_layer
policy_layer (non-user-visible; hash or seal)
```

Loss ledger: when layers are collapsed for token budget, ledger records what was erased (Paper B).

7.6 Provenance Normal Form (PvNF)

A generated answer or derived data product is in **Provenance Normal Form** iff it declares:

```
source_facts, transformations, model_or_tool_calls, uncertainty,  
non_replayable_steps
```

Extends §4.4 Provenance Form (semiring packets) to **model outputs** and **irreversible steps** (sampling, external API, human judgment). Terminal *Refuse* when provenance cannot be reconstructed.

7.7 Freshness Normal Form (FNF)

A query result is in **Freshness Normal Form** iff it carries:

```
read_timestamp, replica_lag, cache_age, invalidation_status,  
freshness_sla
```

Composition: pairs with BQNF *Invalidate* / *Preserve* — FNF exposes *why* a read is stale; BQNF decides *whether* repair is possible.

7.8 Stripe Normal Form (SnNF)

Stripe here means **shard/stripe layout** (erasure coding, parity), not payment processing. Data is in **Stripe Normal Form** iff every logical object has a reconstructable map of:

```
shards, erasure_fragments, parity_layout, encryption_domains,  
failure_groups
```

Use: object storage, distributed filesystems, columnar shards. Excludes claiming durability without stating survived failure modes.

7.9 Authority Normal Form (ANF)

A system object satisfies **Authority Normal Form** iff consumers can distinguish five gates:

Gate	Meaning
<code>available</code>	bits reachable (RPC success, row returned)
<code>valid</code>	satisfies schema, types, local constraints
<code>certified</code>	normal form + certificate + obstruction/terminal declared
<code>authorized</code>	policy/verifier accepts trust lineage
<code>actionable</code>	permitted to drive side effects (write, spend, notify)

Extends §0 hierarchy explicitly for distributed readers:

Each gate is a **separate check**. In particular: `available` does not imply `valid`; `certified` does not imply `authorized` (§0). The classical hierarchy $\text{Form} \subset \text{NF} \subset \text{Certified} \subset \text{Authorized}$ applies to **representation shapes**, not to RPC reachability.

Terminal: `AuthorityLaundering` (Paper B) when a lower gate is mistaken for a higher one.

7.10 Epistemic Normal Form (ENF) — master packet

A distributed AI/data system is in **Epistemic Normal Form** when every outward claim about data can be reduced to a master packet:

```
what_is_known  
by_whom  
as_of_when  
under_which_authority  
with_what_uncertainty  
terminal_route_if_challenged
```

ENF is the **composition target** of LNF–SnNF: subordinate normal forms supply fields; ENF forbids omitting any dimension. Challenge routes reuse

BQNF/GTMUR terminals (*Escalate* , *Invalidate* , *Refuse* , *Recompute*).

Thesis (epistemic replacement for Codd default).

Modern systems should be judged not only on whether dependencies are decomposed, but on whether they can truthfully answer: *where is it, what version is it, who may rely on it, what uncertainty remains, and what happens if challenged?*

Paper program (future): operational checklists and certificates per epistemic NF, analogous to Paper A (BQNF) and Paper B (GTMUR).

7.11 Mapping epistemic NFs to the existing stack

Epistemic NF	Nearest existing artifact
StNF, FNF	BQNF repair + CDC evidence (Paper A)
PvNF	Provenance form (§4.4), certificates (Paper B)
ANF, ENF	Form $\subset$ NF $\subset$ Certified $\subset$ Authorized (§0); GTLA verifier (§5.4)
ONF, CNF	Orchestration certificates (future); GTMUR loss ledger for collapsed context
LNF, RNF, SnNF	Replication/metadata catalogs; Stripe layout manifests

8. Synthesis

- **Shape Normal Forms** exclude static dependency and range anomalies in data at rest.
- **Boundary-Quotient Normal Form (BQNF)** certifies whether runtime data in motion represents a sound repair or an obstruction.

- **Evidence and Certificate Forms** standardize audit trails, provenance, and the structural skeletons preserved during transitions.
- **Authority Contracts** govern the active transport, comparison, and finite shadowing of representation states, ensuring authority descent remains bounded and checked.
- **Epistemic Normal Forms** (§7) specify what a distributed system must declare so it does not pretend to omniscient global state — composing locality, version, freshness, provenance, and authority into bounded claims.

Relvar, Relation, Table, Record, Field — and a Different Kind of Relvar

Status: Archived source / Definitional provenance.

Absorbed base ontology copy from

`omega_engine/research/in_work/simplex_cosmos_papers/SMPLX_009_Simplex`

`Databases/crap/0_relvar_relation_table_and_boundary_relvar.md`. The source folder was integrated into the root scaffold and removed.

Non-normative archive: current definitions are controlled by

`book/00_preface.md` through `book/08_open_problems.md`, `glossary/`, `spec/`, and `contracts/`. If this appendix conflicts with the cleaned taxonomy, the cleaned taxonomy wins.

Status: base ontology / terminology anchor for SMPLX-009

Tone: deliberately strict

Purpose: answer *what exactly is being normalized, repaired, transported, or authorized?* before quotient repair, transport certificates, or epistemic normal forms appear.

Companion papers (built on this vocabulary):

Document	Object in focus
<code>1_normal_forms.md</code>	taxonomy atlas
<code>1a_bqnf_incremental_repair.md</code>	certified transition on a boundary relvar
<code>1b_gtmur_representation_transport.md</code>	certified compilation between relvar regimes
<code>1c_*</code> (planned)	finite-shadow authorization over approximations

0. Why this file exists

0.1 The anchor question

Every later paper risks floating abstractions unless this is settled first:

What is the thing being normalized, repaired, transported, or authorized?

Answer, in order of introduction:

Layer	Thing	Kind
Classical	relvar $\mathcal{R}$	variable ranging over relation values $\mathcal{R}_t$
Motion	boundary relvar $\mathcal{B}$	relvar + admissible transitions and repair semantics
Observation	observer relvar $\mathcal{O}$	relvar + declared visibility / indistinguishability
Evidence	certificates & ledgers	proof artifacts about transitions and compilations
Governance	verifier	separates certified from authorized

Without this file, BQNF and GTMUR are syntax without a subject. With it, the stack has a base ontology.

0.2 Conservative extension (why the order matters)

This note does **not** open with quotient spaces, transport morphisms, epistemic authority, or AI orchestration. It opens with:

domain → attribute → tuple → relation → relvar

Only after that vocabulary is fixed do we extend to boundary relvars and observer relvars. That order earns trust from relational readers: Codd is continued, not discarded.

Database people often say:

table
row
column
field
record
relation

as if those words were interchangeable.

They are not.

That sloppiness matters. Once the terminology is loose, every later idea becomes mush: normal forms, keys, dependency theory, SQL implementation, change repair, cache invalidation, provenance, and authority transport.

So this file begins in the older, stricter relational style: relation, tuple, attribute, domain, relvar. Then it explains why SQL tables, rows, columns, records, and fields are implementation-adjacent terms, not exact logical equivalents.

After that, we introduce a different kind of relvar: not a classical relvar that only ranges over relation values at rest, but a **certified boundary relvar** whose admissible changes are part of its declared type.

This is not a rejection of Codd. It is a continuation of the same discipline applied to data in motion.

Part I — The strict vocabulary

1. Domain

A **domain** is a set of permitted values.

Examples:

```
StudentId
CourseId
MoneyAmount
EmailAddress
Timestamp
```

Mathematically:

$$[D = \{v \mid v \text{ satisfies the domain predicate} \}]$$

A domain is not merely a SQL type.

For example:

```
email varchar(320)
```

is not really an email domain. It is a character string column with a length limit.

A better approximation is:

```
CREATE DOMAIN email_address AS text
CHECK (
    VALUE ~* '^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$'
);
```

PostgreSQL supports `CREATE DOMAIN`. SQL Server does not support domains in the same relational sense; user-defined data types and `CHECK` constraints can approximate some of the behavior, but they do not give a full domain theory.

Curmudgeon rule:

A data type is storage machinery. A domain is a semantic contract.

2. Attribute

An **attribute** is a named role whose values come from a domain.

Example:

```
StudentId : StudentIdDomain  
StudentName : NameDomain  
BirthDate : DateDomain
```

An attribute is not just “a column.”

A column is a common physical or SQL representation of an attribute. But the attribute is part of the logical heading of a relation.

A relation heading is a set of attributes:

[$H = \{A_1:D_1, \dots, A_n:D_n\}$]

Order does not matter.

That is already different from most SQL displays, where columns appear left-to-right.

Curmudgeon rule:

An attribute is logical. A column is representational.

3. Tuple

A **tuple** is a set of attribute-value pairs, one value for each attribute in a heading.

For heading:

[$H = \{\text{StudentId}, \text{CourseId}, \text{Grade}\}$]

a tuple may be:

[$t = \{ \text{StudentId} \mapsto 42, \text{CourseId} \mapsto 1001, \text{Grade} \mapsto 'A' \}$]

A tuple is not a row.

A row is an implementation/display object. It often has an order, can contain `NULL`, may be duplicated in SQL query results, and is processed by the DBMS in ways that are not identical to the mathematical tuple.

A tuple has:

- no duplicate identity
- no physical position
- no hidden row number
- no order of attributes

Curmudgeon rule:

A tuple is a fact-shaped value. A row is how a system happens to show or store something.

4. Relation

A **relation** is a set of tuples with the same heading.

$[R \subseteq \prod_{A_i \in H} D_i]$

or, more precisely:

$[R = (H, B)]$

where:

H = heading

B = body, a set of tuples over H

Because the body is a set:

no duplicate tuples

no tuple order

no attribute order

Example relation value:

ENROLLMENT

Heading:

StudentId : StudentId

CourseId : CourseId

Grade : Grade

Body:

{StudentId=1, CourseId=10, Grade='A'}

{StudentId=2, CourseId=10, Grade='B'}

This relation is a **value**.

That point is crucial.

A relation is like the integer value 7. It is not a storage location. It does not change. If something “changes,” then some variable now holds a different relation value.

Curmudgeon rule:

A relation does not update. A relvar is assigned a new relation value.

Structural hinge. The entire later framework depends on separating:

value	(relation at an instant)
variable	(relvar – name for a sequence of values)
transition	(event $e : R_t \rightarrow R_{\{t+1\}}$)
observer	(projection 0 – what may be distinguished)
authority	(verifier – what may be acted upon)

Most systems discourse collapses these. This project does not.

5. Relvar

A **relvar** is a relation variable.

It is a named variable whose possible values are relations of a declared type.

Enrollment

is a relvar if it can hold different relation values over time, all with the same heading:

[Enrollment_t ⊆ StudentId × CourseId × Grade]

At time (t₀):

[Enrollment := R₀]

At time (t₁):

[Enrollment := R₁]

The relvar persists as a variable. Its value changes.

A base relvar is stored directly. A derived relvar is defined by an expression over other relvars.

Example derived relvar:

[CurrentPaidOrders = $\sigma_{\text{status}='paid'}$ (Orders)]

SQL approximation:

```
CREATE VIEW CurrentPaidOrders AS
SELECT order_id, customer_id, amount
FROM Orders
WHERE status = 'paid';
```

Curmudgeon rule:

The table name in SQL is usually trying to be a relvar name. It only partly succeeds.

6. Table

A **table** is a representation.

In SQL practice, a table often acts like a relvar, but it is not identical to a relation or relvar.

Reasons:

1. SQL tables may allow duplicate rows unless constrained.
2. SQL has **NULL** ; the classical relational model does not treat null as an ordinary domain value.
3. SQL columns are ordered in metadata and output.
4. SQL result sets may be bags, not sets.
5. SQL tables may contain implementation artifacts: identity columns, rowversion columns, computed columns, sparse columns, system-period columns.
6. SQL constraints are not the same thing as full relational predicates.

Example:

```
CREATE TABLE Enrollment (  
    student_id int NOT NULL,  
    course_id int NOT NULL,  
    grade char(2) NULL  
);
```

This is not yet a good relvar approximation.

Better:

```
CREATE TABLE Enrollment (  
    student_id int NOT NULL,  
    course_id int NOT NULL,  
    grade char(2) NOT NULL,  
    CONSTRAINT PK_Enrollment PRIMARY KEY (student_id, course_id),  
    CONSTRAINT CK_Enrollment_Grade CHECK (grade IN  
    ('A', 'B', 'C', 'D', 'F'))  
);
```

Closer, but still SQL.

Curmudgeon rule:

A SQL table is not a relation. It is an industrial approximation to a relvar, with compromises.

7. Record

A **record** is a programming or storage structure: a compound value with named or positional fields.

Example in C#:

```
public record EnrollmentRecord(int StudentId, int CourseId, string  
    Grade);
```

Example in JSON:

```
{  
  "student_id": 1,  
  "course_id": 10,  
  "grade": "A"  
}
```

A record may represent a tuple, but it is not the same thing as a tuple.

A record can:

- have field order
- have optional fields
- have missing fields
- have null fields
- have nested structure
- carry object identity
- exist outside a relation

A tuple in the relational sense is a value over a heading.

Curmudgeon rule:

A record is a programming object. A tuple is a relational value.

8. Field

A **field** is usually a component of a record.

In casual SQL speech, people say “field” when they mean column or attribute. That habit is common. It is also imprecise.

There are at least three different things people call “field”:

Casual phrase	More precise term
field in a table	column, or better, attribute
field in a row	scalar value at an attribute in a tuple/row
field in a record	component of a record structure

In a relation, the closest formal idea is:

[t(A)]

the value of attribute (A) in tuple (t).

Example:

[t(StudentName) = 'Ada']

Curmudgeon rule:

If you mean attribute, say attribute. If you mean column, say column. If you mean record component, say field.

9. Column

A **column** is a tabular representation of an attribute.

SQL column:

```
student_id int NOT NULL
```

Relational attribute:

[StudentId : StudentIdDomain]

A column can approximate an attribute if:

- it has a declared type/domain
- it rejects nulls when the model requires total values
- it participates in the correct constraints
- it is understood as unordered with respect to other attributes

Curmudgeon rule:

Columns are what SQL gives you. Attributes are what the relational model talks about.

10. Row

A **row** is a tabular representation of a tuple.

A SQL row may be:

- ordered by display position
- duplicated in a result set
- partially unknown through NULL
- extended with hidden system fields
- physically stored, moved, locked, versioned, or indexed

A relational tuple is a logical value.

Curmudgeon rule:

Rows are implementation artifacts. Tuples are logical facts.

Part II — A compact dictionary

Strict relational term	SQL-ish term	Programming-ish term	Warning
domain	data type / domain / check constraint	type	SQL type is not full semantic domain
attribute	column	property / field name	attribute is logical, column is representation
tuple	row	record instance	tuple is unordered and set-valued
relation	table value / result set	collection of records	SQL result may be a bag, not a set
relvar	base table / view name	variable / repository	relvar is a variable over relation values
relation predicate	table meaning	invariant / contract	often missing from SQL DDL
key	primary/unique key	identity constraint	key is logical uniqueness, not surrogate decoration
foreign key	FK constraint	reference	FK approximates inclusion dependency
null	NULL	null / None	not a value in the classical relational sense

Part III — What a relvar really declares

A serious relvar declaration should contain more than a SQL table name.

It should declare:

- name
- heading
- domains
- candidate keys
- foreign keys / inclusion dependencies
- relation predicate
- allowed updates
- derived dependencies
- integrity constraints

Example:

Relvar: Enrollment

Heading:

- StudentId : StudentId
- CourseId : CourseId
- Grade : Grade

Predicate:

Student StudentId is enrolled in Course CourseId and currently has Grade.

Candidate key:

{StudentId, CourseId}

Dependencies:

{StudentId, CourseId} -> Grade

Foreign keys:

- StudentId references Student
- CourseId references Course

SQL approximation:

```
CREATE TABLE Enrollment (  
    student_id int NOT NULL,  
    course_id int NOT NULL,  
    grade char(2) NOT NULL,  
    CONSTRAINT PK_Enrollment PRIMARY KEY (student_id, course_id),  
    CONSTRAINT CK_Enrollment_Grade CHECK (grade IN  
( 'A', 'B', 'C', 'D', 'F' )),  
    CONSTRAINT FK_Enrollment_Student FOREIGN KEY (student_id)  
        REFERENCES Student(student_id),  
    CONSTRAINT FK_Enrollment_Course FOREIGN KEY (course_id)  
        REFERENCES Course(course_id)  
);
```

The SQL is useful. It is not the whole declaration.

The missing piece is the predicate:

Student `student_id` is enrolled in course `course_id` and has grade `grade` .

That sentence is not decoration. It is the meaning of the relvar.

Part IV — Why this matters for normal forms

Normal forms are not table beautification.

A normal form is a discipline on relvars.

Classical normal forms ask:

Can dependency anomalies hide inside this relvar?

Examples:

[student_id \to student_name]

does not belong inside:

[Enrollment(student_id, course_id, student_name, course_title, grade)]

because the relvar is mixing at least three predicates:

```
student_id names student_name
course_id names course_title
student_id is enrolled in course_id with grade
```

The repair is not cosmetic. It separates predicates:

```
Student(student_id, student_name)
Course(course_id, course_title)
Enrollment(student_id, course_id, grade)
```

Equation:

[EnrollmentBad \cong Student \Join Enrollment \Join Course]

when the dependencies are true and the decomposition is lossless.

Curmudgeon rule:

Bad normalization usually means several predicates are being stuffed into one relvar.

Part V — SQL does not rescue bad thinking

SQL can enforce some relational discipline:

PRIMARY KEY
UNIQUE
FOREIGN KEY
CHECK
NOT NULL
EXCLUDE, in PostgreSQL
triggers, with caution

But SQL cannot infer your intended predicate.

This table is legal:

```
CREATE TABLE Thing (  
    id int PRIMARY KEY,  
    name text NOT NULL,  
    amount numeric(12,2) NOT NULL,  
    status text NOT NULL  
);
```

It is also mostly meaningless.

A relvar without a predicate is just a bucket with constraints.

Curmudgeon rule:

DDL without a predicate is not design. It is furniture assembly.

Part VI — The bridge to our work

Classical relational theory disciplines **data at rest**.

It asks whether relation values are shaped so that dependency anomalies do not hide.

Our newer work asks a different question:

When a relvar changes, what else is allowed to change,
what can be repaired,
what must be invalidated,
and what authority survives the transition?

This is the movement from:

[R_t]

to:

[$R_t \xrightarrow{e} R_{t+1}$]

A classical relvar declares the admissible relation values.

A certified change-aware relvar declares admissible relation values **and**
admissible transitions.

The key distinction from the current framework is:

[$\text{Form} \subset \text{Normal Form} \subset \text{Certified Normal Form}$
 $\subset \text{Authorized Normal Form}$]

where a certified normal form must carry a residual obstruction, proof artifact,
and terminal route. This hierarchy is the guardrail against calling every
convenient representation a normal form.

Unifying invariant (across the whole stack)

Category theory, quotients, and obstruction calculus are **tools**. The recurring
invariant is:

[$\boxed{\text{No silent authority degradation.}} \]$

Everything else names a mechanism:

Mechanism	What it forbids silently
BQNF / boundary relvar	uncertified repair, stale cache as fresh

Mechanism	What it forbids silently
GTMUR / CpNF	erasure without ledger, compile drift
loss ledger	forgotten distinctions
observer boundary	acting on invisible attributes as if seen
repair terminals	pretending knowledge when evidence is missing
certified vs authorized	treating validity as permission
finite shadows (GTTC)	limit objects without approximating towers

Systems must declare where authority was lost — or prove it was preserved.

Scope discipline (what may enter the framework)

The framework can absorb AI, streams, vectors, and federation — but only under this admission rule:

[$\boxed{\text{Can this anomaly be detected, typed, and terminalized with finite evidence?}}$]

- **Yes** → operational normal form (BQNF, CpNF, evidence packets, verifier terminals).
- **No** → philosophical / speculative layer (atlas §7 epistemic NFs until checklists exist).

This criterion prevents abstraction drift: a theory that explains everything operationalizes nothing.

Philosophical shift (post-relational, not anti-relational)

Classical relational theory assumes **global truth** at a snapshot: one admissible relation value R_t .

This work assumes **bounded observable authority**: what observer O may rely on, as of a declared time, under a policy, with explicit uncertainty.

Not a rejection of relations. A **runtime semantics layer** for distributed authoritative computation — *relational transition semantics with certified observer boundaries*.

Part VII — A different kind of relvar

1. Classical relvar

A classical relvar is:

$[\text{Rel} : \text{Time} \rightarrow \text{Rel}(H)]$

At each time:

$[\text{Rel}(t) = R_t]$

where (R_t) is a relation over heading (H) .

This says:

at time t , this relvar holds this relation value

It does not, by itself, say whether a particular update can safely repair a cache, preserve a materialized view, commute with another update, or transport authority.

2. Boundary relvar

A **boundary relvar** is a relvar with an explicit boundary for change.

It declares:

- relation heading
- ordinary integrity constraints
- change event type
- boundary map
- repair image
- obstruction quotient
- terminal routes

Mathematically:

$$[\mathcal{B} = (H, \Sigma, C^0 \xrightarrow{d_0} C^1, Q^1, \text{Term})]$$

where:

$$[Q^1 = C^1 / \text{im}(d_0)]$$

A write event (e) induces a defect:

$$[\omega(e) \in C^1]$$

The event is exactly repairable iff:

$$[\omega(e) = 0 \in Q^1]$$

equivalently:

$$[\omega(e) = d_0 \alpha]$$

for some admissible repair (α).

If not, the relvar must terminalize:

- Invalidate
- Serialize
- Recompute
- Lift
- Unsupported
- ConservativeInvalidate
- Escalate
- Refuse

This follows the boundary-certified database idea: a database should not guess what a write breaks; it should compute the obstruction.

Evolution in one line: Codd disciplined **valid states**; boundary relvars discipline **valid transitions and certified repairs**.

[$\underbrace{\text{Rel}(H)}_{\text{states at rest}} \quad \leadsto \quad \underbrace{(C^0 \rightarrow_{d_0} C^1, Q^1, \text{Term})}_{\text{transitions in motion}}$]

Full operational treatment: `1a_bqnf_incremental_repair.md`.

3. Why this is still relational

This is not replacing relations with vibes.

The ordinary relation is still present:

[$R \subseteq \prod_{A \in H} D_A$]

The new structure is attached to transitions:

[$R_t \rightarrow \{e\} R_{t+1}$]

The classical relvar says what states are valid.

The boundary relvar says which state transitions are certified, repairable, or terminal.

Plainly:

Classical relvar:

What facts are currently true?

Boundary relvar:

What facts are currently true, and what did this change break?

Part VIII — Example: ordinary relvar versus boundary relvar

1. Ordinary base relvar

```
CREATE TABLE Orders (  
    order_id bigint PRIMARY KEY,  
    customer_id bigint NOT NULL,  
    amount numeric(18,2) NOT NULL CHECK (amount >= 0),  
    status text NOT NULL CHECK (status IN ('open','paid','void'))  
);
```

Relational predicate:

Order order_id belongs to customer customer_id,
has monetary amount amount,
and currently has lifecycle status status.

Relation type:

[Orders \subseteqq OrderId \times CustomerId \times Money \times Status]

2. Derived relvar

```
CREATE VIEW PaidCustomerSpend AS  
SELECT  
    customer_id,  
    COUNT(*) AS order_count,  
    SUM(amount) AS gross_sum,  
    AVG(amount) AS avg_order  
FROM Orders  
WHERE status = 'paid'  
GROUP BY customer_id;
```

Algebra:

```
[ Q = \gamma_{customer\_id;\ COUNT(*), SUM(amount), AVG(amount)}  
(\sigma_{status='paid'}(Orders)) ]
```

This is a derived relvar: a relation variable whose value is determined by another relvar expression.

3. Boundary declaration

Boundary fingerprint:

```
[ \partial Q = ( Orders, {customer\_id,amount,status}, status='paid', GROUP BY\  
customer\_id, {COUNT,SUM,AVG} ) ]
```

That says which part of `Orders` can affect the derived relvar.

If an update changes only an irrelevant column, the view does not care.

If an update changes `amount`, `status`, or `customer\_id`, the boundary is touched.

4. Delta equations

For a write event (e) with old row (o) and new row (n):

```
[ \Delta SUM = 1_{n.status='paid'}n.amount  
1_{o.status='paid'}o.amount ]
```

[Δ COUNT = 1_

$1_{\{o.status='paid'\}}$]

$[AVG' = \frac{SUM + \Delta SUM}{COUNT + \Delta COUNT}]$

The repair is exact when the event evidence contains:

old amount
new amount
old status
new status
old customer_id
new customer_id

If that evidence is missing, the system must not pretend.

5. Certified boundary relvar declaration

Pseudo-DDL:

```

CREATE BOUNDARY RELVAR PaidCustomerSpend_Boundary
ON VIEW PaidCustomerSpend
SOURCE Orders
BOUNDARY (
    attributes: customer_id, amount, status,
    predicate: status = 'paid',
    group_key: customer_id,
    aggregates: count, sum, avg
)
REPAIR IMAGE (
    count: delta_count(old.status, new.status),
    sum: delta_sum(old.status, new.status, old.amount, new.amount),
    avg: sum / count
)
EVIDENCE REQUIRED (
    old.customer_id,
    new.customer_id,
    old.status,
    new.status,
    old.amount,
    new.amount
)
TERMINALS (
    Repair,
    Invalidate,
    Recompute,
    ConservativeInvalidate,
    Unsupported
);

```

That is not standard SQL. It is the shape of the missing declaration.

6. SQL approximation

Cache table:

```
CREATE TABLE CustomerSpendCache (  
    customer_id bigint PRIMARY KEY,  
    order_count bigint NOT NULL,  
    gross_sum numeric(18,2) NOT NULL,  
    avg_order numeric(18,2) NOT NULL,  
    last_repaired_at timestamp NOT NULL DEFAULT current_timestamp  
);
```

Repair certificate:

```
CREATE TABLE RepairCertificate (  
    certificate_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    source_relvar text NOT NULL,  
    target_relvar text NOT NULL,  
    event_key text NOT NULL,  
    obstruction_class text NOT NULL,  
    terminal_route text NOT NULL CHECK (  
        terminal_route IN (  
            'Repair',  
            'Invalidate',  
            'Serialize',  
            'Recompute',  
            'Lift',  
            'Unsupported',  
            'ConservativeInvalidate',  
            'Escalate',  
            'Refuse'  
        )  
    ),  
    evidence_json text NOT NULL,  
    created_at timestamp NOT NULL DEFAULT current_timestamp  
);
```

The table stores the evidence packet. It does not magically prove the repair.
The verifier does that.

Curmudgeon rule:

A certificate table without a verifier is just another table.

Part IX — Another different kind: observer relvar

A derived relvar can also declare what it ignores.

For example:

```
CREATE VIEW CustomerNames AS
SELECT customer_id, display_name
FROM Customer;
```

This observer sees:

```
customer_id
display_name
```

It ignores:

```
email
phone
credit_limit
internal_notes
```

Observer equivalence:

$$[e_1 \sim_{\{\text{mathcal O}\}} e_2]$$

when both events produce the same visible effect under observer (mathcal O).

Functorially:

$$[\text{mathcal O} : \text{Rel}(H) \rightarrow \text{Rel}(H_{\{\text{mathcal O}\}})]$$

The observer relvar declares the kernel of that projection:

$$[\ker(\text{mathcal O})$$

$$\{(r_1, r_2) \mid \text{mathcal O}(r_1) = \text{mathcal O}(r_2)\}$$

Plain English:

This relvar knows what it cannot see.

SQL approximation:

```
CREATE TABLE ObserverBoundary (  
    observer_name text NOT NULL,  
    source_relvar text NOT NULL,  
    visible_attribute text NOT NULL,  
    PRIMARY KEY (observer_name, source_relvar, visible_attribute)  
);
```

Example:

```
INSERT INTO ObserverBoundary(observer_name, source_relvar,  
    visible_attribute)  
VALUES  
    ('CustomerNames', 'Customer', 'customer_id'),  
    ('CustomerNames', 'Customer', 'display_name');
```

A change to `email` does not intersect the observer boundary. A change to `display_name` does.

This is not a classical relvar. It is a relvar plus declared observational authority.

Observational Normal Form (candidate)

An **observer relvar** is in **Observational Normal Form** when every admissible representation declares:

```
observer identity  
visible heading H_0  
ignored attributes (kernel of projection)  
equivalence:  $e_1 \sim_0 e_2$  iff indistinguishable to 0  
intersection rule for change events
```

Anomaly excluded: treating observer-relative state as if it were global truth.

Plain English for $e_1 \sim_{\mathcal{O}} e_2$:

changes indistinguishable to observer $\mathcal{O}$

Connections (operational, not metaphorical):

Domain	Role of $e \sim_{\mathcal{O}} e'$
cache / MV	disjoint column change $\rightarrow$ preserve
CQRS	read model sees subset of write stream
RBAC	policy-masked attributes
federated views	remote columns invisible to local observer
differential privacy	indistinguishability under noise
AI context window	retrieved vs hidden fields (ledger when collapsed)

Observer relvars may outlive any single transport acronym because modern systems are **observer-relative** by construction. GTMUR compiles between regimes; observers declare what each regime may distinguish.

Part X — What not to say

Do not say:

A relation is a table.
A tuple is a row.
An attribute is a column.
A field is a column.
A record is a row.

Those are compromises, not definitions.

Better:

A SQL table usually attempts to implement a relvar.
 A SQL row often represents a tuple.
 A SQL column often represents an attribute.
 A record may encode a tuple.
 A field may encode an attribute-value component.

That language is less convenient. It is also less wrong.

Part XI — Final kernel

[$\boxed{\text{Relation} = \text{set of tuples over a heading}}$]

[$\boxed{\text{Relvar} = \text{variable whose values are relations}}$]

[$\boxed{\text{Table} = \text{SQL representation that may approximate a relvar}}$]

[$\boxed{\text{Tuple} \neq \text{row}}$]

[$\boxed{\text{Attribute} \neq \text{column}}$]

[$\boxed{\text{Field} \neq \text{attribute}}$]

[$\boxed{\text{Record} \neq \text{tuple}}$]

And the extension:

[$\boxed{\text{text}}$

$\text{relvar} + \text{declared transition boundary} + \text{repair quotient} + \text{terminal route}$]

A classical relvar disciplines facts at rest.

A boundary relvar disciplines facts in motion.

The serious version of the project is not “let us invent cute names.” It is:

make every hidden dependency,
hidden boundary,
hidden repair assumption,
and hidden authority claim explicit.

That is the professional standard.

Part XII — Stack map (what this file enables)

Question	Subject	Primary artifact
What are the facts?	relation value R_t	classical NF, keys, dependencies
What may change?	relvar assignment $\mathcal{R}(t) := R_t$	transactions, constraints
What did a write break?	boundary relvar B , defect $\omega(e)$	BQNF, Paper A
Was compilation honest?	morphism $F:A \rightarrow A_{\mathrm{nf}}$, witness χ_F	GTMUR / CpNF, Paper B
What may this observer see?	observer relvar $\mathcal{O}$, $\sim_{\mathcal{O}}$	Observational NF, §IX
May we trust this approximation?	finite shadow tower	GTTC, Paper C (planned)
May we act on it?	authority verifier	certified $\neq$ authorized

Landing position (publishable framing):

Relational transition semantics with certified observer boundaries.

Less grandiose than “replacing Codd”; more accurate for distributed caches, replicas, compiled views, and agent-mediated writes.

Implementation north star: resist infinite extensibility — ship finite-evidence verifiers, real certificates, failure pathologies, and repair-vs-recompute benchmarks first ([1a §9](#), [1b §9](#)). Ontology follows executable proof.

General Theory of Observers and Distinguishability: Domain Extensions and Observer Contracts

Status: Archived source / Domain extension.

Absorbed observer/domain-extension copy from
`omega_engine/research/in_work/simplex_cosmos_papers/SMPLX_009_Simplex Databases/normal_form_contracts/reInfnf.md`.

Non-normative appendix: current definitions are controlled by
`book/00_preface.md` through `book/08_open_problems.md`, `glossary/`,
`spec/`, and `contracts/`. This file is retained for provenance and domain-
extension vocabulary. If it conflicts with the protected-NF rule or
quarantine policy, the cleaned taxonomy wins.

This document maps the underlying mathematical substrate of observers,
distinguishability, and coordinates. To prevent taxonomy inflation, these are
classified as **Observer Contracts** and **Domain Extensions** rather than
database normal forms.

1. Thesis and Master Questions

Classical normal forms prevent structural anomalies in stored relations. Simplex normal forms prevent authority anomalies in data that moves, transforms, replicates, summarizes, delegates, or becomes evidence for action.

Under this architecture, difference is primitive; sameness is observer-relative certified zero-difference. The common spine across databases, distributed systems, networks, and physics is built on these master questions:

1. **What is distinguishable?** What measurements separate two states?
 2. **What counts as same?** What equivalence relation identifies them?
 3. **What survives comparison?** What survives transport between observers?
 4. **What fails to glue?** Can local observer sections form a global section?
 5. **What can be repaired?** Can the defect be resolved by allowed corrections?
 6. **What residual remains?** What remains after repair and comparison?
 7. **What authority is earned?** What finite terminal certificate is earned?
-

2. Observer Contracts

Observer contracts define the interface through which representation changes and observations are compared.

2.1 Observer Distinguishability Contract (ODC)

Formerly called ODNF; this contract models the base questions of state separation and transport:

- **Distinguishable:** Separated by observable cochain defects $\omega \in C^1$.
- **Same:** Equivalence under repairs $d_0\alpha$.
- **Comparison:** Preservation under transport map χ .
- **Gluing:** Existence of a global section on the observer sheaf.

Finite Obstruction Realization:

- C^0 : Allowed repairs or gauge transformations.
- C^1 : Observer defects or distinguishability gaps.

- C^2 : Gluing failures or contextuality failures.
- **Quotient Space:** $Q^1 = C^1 / \sim$.
- **Obstruction Class:** $[\omega] \in Q^1$.

2.2 Observer Information Contract (OIC)

Formerly called SINP; this contract governs signal transmission over noisy boundaries:

- **Alphabet:** Declared message domain.
- **Channel:** Noisy transport characteristics.
- **Distinguishability:** Bounded by mutual information and error probability.
- **Residual:** Unresolved entropy recorded in a loss ledger.

Terminal Routes:

- `Decodable` : Signal successfully reconstructed.
 - `Ambiguous` : Multiple valid decodings exist.
 - `CapacityExceeded` : Channel capability bounds violated.
 - `Quarantine` : Isolate packet to prevent erroneous interpretation.
 - `Decompose` : Split composite message into independently decodable parts.
 - `Refuse` : Decline representation due to excessive noise.
-

3. Domain Extensions

Domain extensions apply the observer stack to specific physical or network contexts. They are quarantined from the core database papers.

3.1 Bell Contextuality Contract (BCC)

Formerly called BCNF_{qm} in source notes; the cleaned registry uses BellCNF to avoid confusion with Boyce-Codd Normal Form. This extension models the failure of classical observer-gluing on local measurement sections:

$\text{local hidden sameness} \neq \text{quantum observational sameness}$

- **Local Sections:** Outcome distributions per measurement context.
- **Global Candidate:** Joint hidden variable assignment.
- **Gluing Failure:** Bell inequality violation indicating a contextual obstruction.

Terminal Routes:

- **ClassicalGlue** : Local contexts merge into a single global section.
- **QuantumSurvivor** : Nonlocal residual verified under quantum assumptions.
- **BellViolation** : Incompatible local margins.
- **Quarantine** : Isolate contextuality failures.

3.2 Spacetime Observer Contracts

Formerly called SRNF and its sub-forms; these govern systems operating over relativistic intervals (e.g., GPS timing, satellite cache replicas, orbital links):

- **Timelike Interval:** Causal order survives all reference frames.
- **Lightlike Interval:** Signal propagation limit boundary.
- **Spacelike Interval:** Order is frame-relative; consensus is required for total sequencing.

A. Reference Frame Contract

- **Declarations:** Observer frame, clock basis, coordinate system, synchronization protocol.
- **Validation:** No timestamp accepted without a declared clock source and conversion witness.
- **Terminals:** `FrameAccepted`, `DriftExceeded`, `FrameMismatch`, `OrderingUnknown`, `Quarantine`.

B. Light-Cone Causality Contract

- **Declarations:** Event location, propagation model, signal speed limit.
- **Validation:** Claimed cause must lie in the past light cone.
- **Terminals:** `CausallyAdmissible`, `SpacelikeConcurrent`, `CausalViolation`, `ConsensusRequired`, `Refuse`.

C. Orbital Placement Contract

- **Declarations:** Spacecraft ID, ephemeris epoch, contact windows, handoff policy.
- **Validation:** Telemetry and replica states mapped to ephemeris contact timelines.
- **Terminals:** `InContact`, `OutOfContact`, `EphemerisStale`, `HandoffPending`, `LocationUnknown`.

D. Clock-Synchronization Contract

- **Declarations:** Clock type, sync source, drift bounds, monotonicity rule.
- **Validation:** Clock bounds must be verified; resets must be logged.
- **Terminals:** `TimeCertified`, `TimeIntervalOnly`, `DriftExceeded`, `ClockReset`, `Quarantine`.

3.3 Lieb-Robinson Locality Contract

This extension captures the finite operational content of locality bounds: a local event cannot be treated as globally available without either a declared propagation cone or an explicit outside-cone residual ledger.

- **Declarations:** Finite locality graph, graph metric, local dependency radius, velocity bound, decay/error bound, event support, observation window.
- **Validation:** Claimed affected support lies inside the locality cone, or the outside-cone effect is terminalized with a residual bound.
- **Terminals:** `WithinLocalityCone`, `ExponentiallySuppressed`, `OutsideConeResidual`, `LocalityBoundExceeded`,

`RefuseInstantInfluence` .

Database reading: invalidation, replication, and agent-memory effects are not omnipresent. Authority spreads only through declared dependency paths.

3.4 Horizon Emission Contract

This extension captures Hawking-radiation-shaped boundary accounting without claiming a physics result. It applies whenever data crosses a boundary after which the interior state is inaccessible: redaction, eviction, compaction, tombstoning, lossy summaries, or exterior-only telemetry.

- **Declarations:** Horizon boundary, interior/exterior partition, crossing rule, emission channel, coarse-graining map, conservation/loss ledger, recoverability class.
- **Validation:** Exterior packets authorize only what the emission channel preserves; interior claims require a lift witness or terminalize as inaccessible.
- **Terminals:** `ExteriorEmissionCertified` , `LossLedgerRequired` , `InteriorInaccessible` , `RecoverabilityUnknown` , `ConservationViolation` , `RefuseUnledgedHorizon` .

Database reading: a compacted aggregate, redacted row, or emitted summary is not authority for erased microstate unless the finite lift is certified.

4. Clean Stack Alignment (Observer Authority Stack)

The Observer Authority Stack (OAS) coordinates the active checks from raw defects to final authorization:

Layer	Question	Output
GTFOC	What defect remains after repair?	Obstruction class $\omega \in Q^1$
GTFDA	Did evidence descend to a terminal?	Terminal authority / refusal
GTMUR	Did representation change preserve authority?	Transport certificate / loss ledger
GTOR	Can partial views be compared?	Comparison envelope / no-glue
GTTC	Do finite shadows authorize continuum claims?	Finite shadow authority
GTTA	What residual remains after all closure attempts?	Typed residual terminal
GTLA	Were all obligations conserved?	Layered authority classification